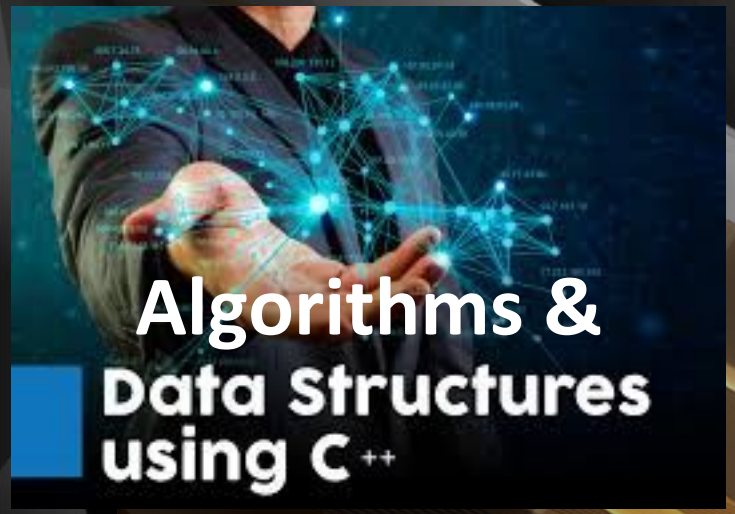


U3 Object-Oriented Programming, Patterns, Data Structures

C++ & Python

Patrones de Diseño de Software

Pattern (C++ & Python Examples)

Introducción a los Patrones de Software

Software Patterns

Patrones de Diseño de Software

Clasificación de los patrones

Los patrones de diseño varían en su complejidad, nivel de detalle y escala de aplicabilidad al sistema completo que se diseña. Me gusta la analogía de la construcción de carreteras: puedes hacer más segura una intersección instalando semáforos o construyendo un intercambiador completo de varios niveles con pasajes subterráneos para peatones.
Los patrones más básicos y de más bajo nivel suelen llamarse *idioms*. Normalmente se aplican a un único lenguaje de programación.
Los patrones más universales y de más alto nivel son los *patrones de arquitectura*. Los desarrolladores pueden implementar estos patrones prácticamente en cualquier lenguaje. Al contrario que otros patrones, pueden utilizarse para diseñar la arquitectura de una aplicación completa.

Además, todos los patrones pueden clasificarse por su *propósito en* tres grupos generales de patrones:

Patrones de creación	Patrones estructurales	Patrones de comportamiento
proporcionan mecanismos de **creación de objetos** que incrementan la flexibilidad y la reutilización de código existente.	explican cómo **ensamblar objetos y clases en estructuras más grandes** a la vez que se mantiene la flexibilidad y eficiencia de la estructura.	se encargan de **una comunicación efectiva y la asignación de responsabilidades** entre objetos.

Patrones de Diseño de Software

Patrones de programación

Patrones de creación

Proporcionan mecanismos de **creación de objetos** que incrementan la flexibilidad y la reutilización de código existente.

2 Patrones de creación
2.1 Constructor
2.2 Fábrica
2.3 Fábrica abstracta
2.4 Prototipo
2.5 Singleton

Patrones estructurales

Explican cómo **ensamblar objetos y clases en estructuras más grandes** a la vez que se mantiene la flexibilidad y eficiencia de la estructura.

3 Patrones estructurales
3.1 Adaptador
3.2 Puente
3.3 Compuesto
3.4 Decorador
3.5 Fachada
3.6 Peso mosca
3.7 Proxy
3.8 Plantilla curiosamente recurrente
3.9 Programación basada en interfaces (IBP)

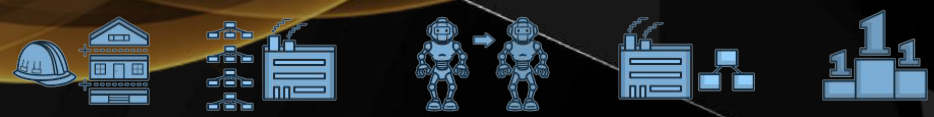
Patrones de comportamiento

Se encargan de **una comunicación efectiva y la asignación de responsabilidades** entre objetos.

4 Patrones de comportamiento
4.1 Cadena de responsabilidad
4.2 Mandar
4.3 Intérprete
4.4 Iterador
4.5 Mediador
4.6 Recuerdo
4.7 Observador
4.8 Estado
4.9 Estrategia
4.10 Template (método)
4.11 Visitante
4.12 Modelo-Vista-Controlador (MVC)

Patrones de creación

Patrones de creación (o creacionales). Definen cómo puede crearse un objeto. Habitualmente esto incluye aislar los detalles de la creación del objeto, de forma que su código no dependa de los tipos de objeto que hay y, por lo tanto, no deba ser modificado al añadir un nuevo tipo de objeto.



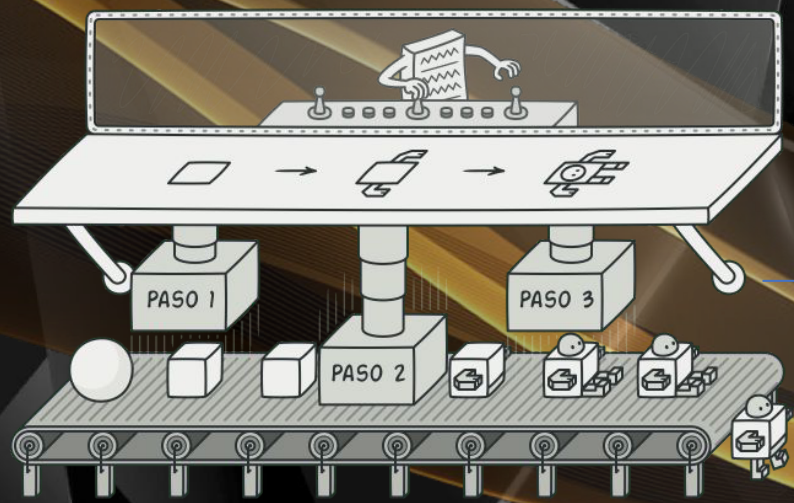
Creational Patterns

Builder (Constructor)

Abstract Factory
(Fábrica Abstracta)

Prototype
(Prototipo)

Singleton
(Instancia Unica)



Patrones de creación

Builder (Constructor)

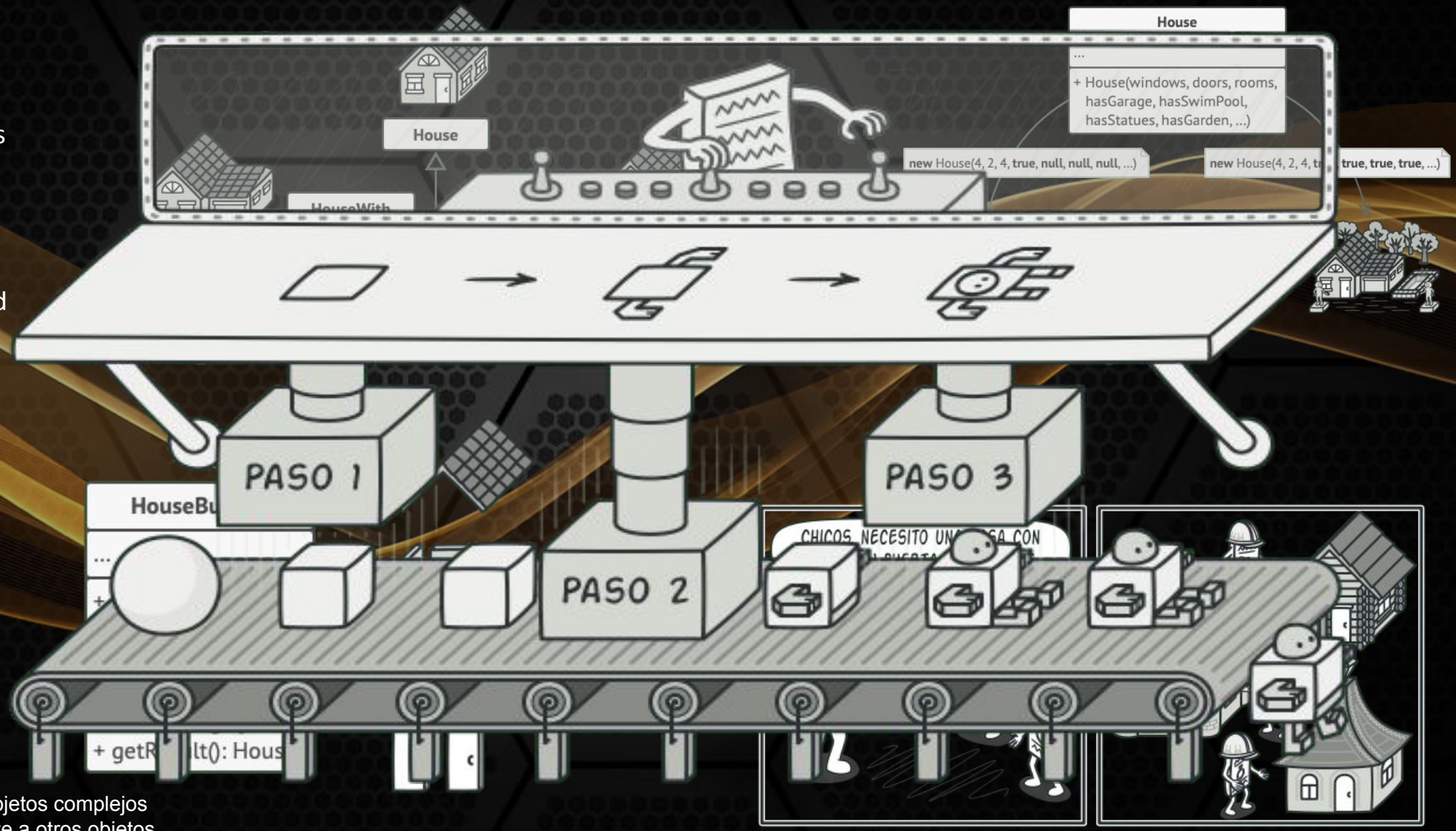
Builder - También llamado: Constructor

Problema

Imagina un objeto complejo que requiere una inicialización laboriosa, paso a paso, de muchos campos y objetos anidados. Normalmente, este código de inicialización está sepultado dentro de un monstruoso constructor con una gran cantidad de parámetros. O, peor aún: disperso por todo el código cliente.

Solución

El patrón Builder sugiere que saques el código de construcción del objeto de su propia clase y lo coloques dentro de objetos independientes llamados constructores.



El patrón Builder te permite construir objetos complejos paso a paso. El patrón Builder no permite a otros objetos acceder al producto mientras se construye.

Patrones de creación – **Builder** (Constructor)

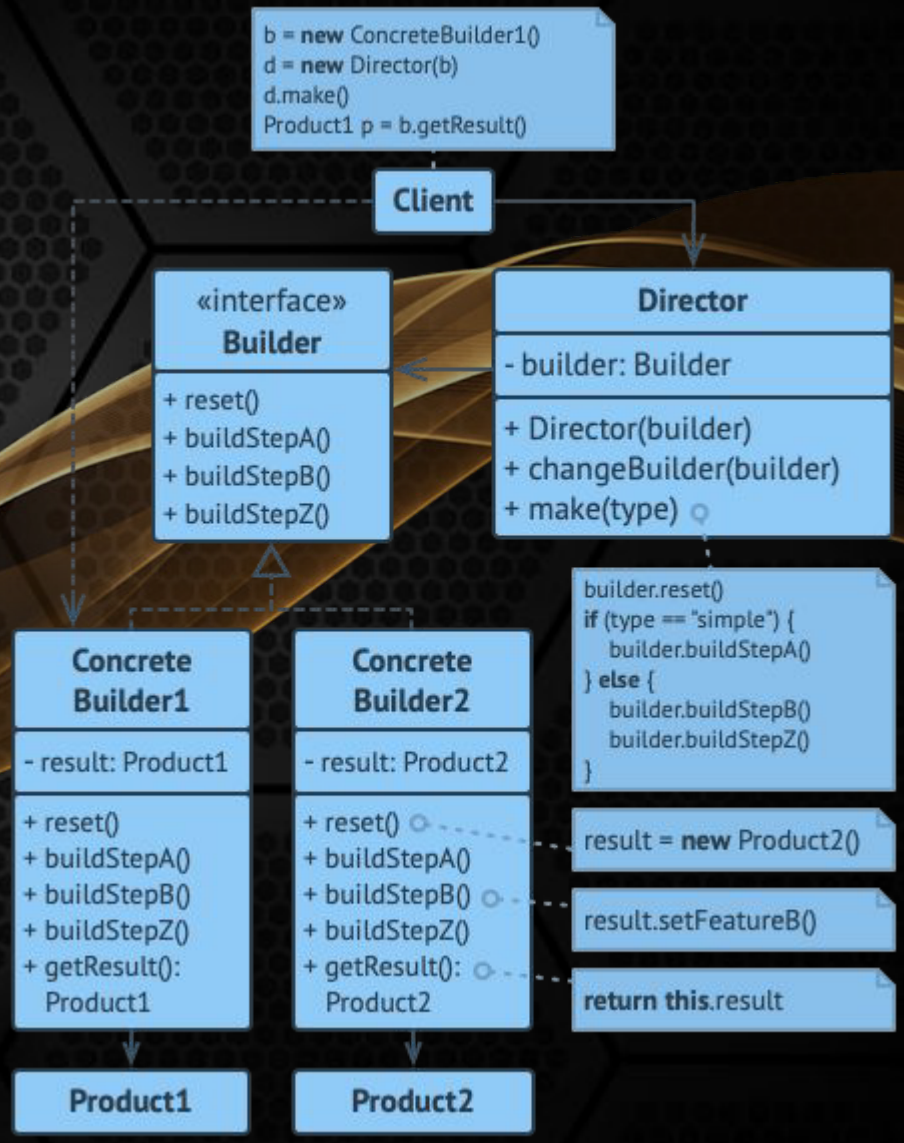
1. La interfaz **Constructora (Builder)** declara pasos de construcción de producto que todos los tipos de objetos constructores tienen en común.

2. Los **Constructores Concretos (Concrete Builder)** ofrecen distintas implementaciones de los pasos de construcción. Los constructores concretos pueden crear productos que no siguen la interfaz común.

3. Los **Productos** son los objetos resultantes. Los productos construidos por distintos objetos constructores no tienen que pertenecer a la misma jerarquía de clases o interfaz.

4. La clase **Director (Director)** define el orden en el que se invocarán los pasos de construcción, por lo que puedes crear y reutilizar configuraciones específicas de los productos.

5. El **Cliente** debe asociar uno de los objetos constructores con la clase directora. Normalmente, se hace una sola vez mediante los parámetros del constructor de la clase directora, que utiliza el objeto constructor para el resto de la construcción. No obstante, existe una solución alternativa para cuando el cliente pasa el objeto constructor al método de producción de la clase directora. En este caso, puedes utilizar un constructor diferente cada vez que produzcas algo con la clase directora.



Patrones de creación – **Builder** (Constructor)

Aplicabilidad

Utiliza el patrón Builder para evitar un "constructor telescópico".
Digamos que tenemos un constructor con diez parámetros opcionales. Invocar a semejante bestia es poco práctico, por lo que sobrecargamos el constructor y creamos varias versiones más cortas con menos parámetros. Estos constructores siguen recurriendo al principal, pasando algunos valores por defecto a cualquier parámetro omitido.

```
class Pizza {
    Pizza(int size) { ... }
    Pizza(int size, boolean cheese) { ... }
    Pizza(int size, boolean cheese, boolean pepperoni) { ... }
    // ...
  // Crear un monstruo semejante sólo es posible en lenguajes
que soportan la sobrecarga de métodos, como C++ C# o Java.
```

El patrón Builder permite construir objetos paso a paso, utilizando tan solo aquellos pasos que realmente necesitamos. Una vez implementado el patrón, ya no hará falta apiñar decenas de parámetros dentro de los constructores.

Utiliza el patrón Builder cuando quieras que el código sea capaz de crear distintas representaciones de ciertos productos (por ejemplo, casas de piedra y madera).
El patrón Builder se puede aplicar cuando la construcción de varias representaciones de un producto requiera de pasos similares que sólo varían en los detalles.
La interfaz constructora base define todos los pasos de construcción posibles, mientras que los constructores concretos implementan estos pasos para construir representaciones particulares del producto. Entre tanto, la clase directora guía el orden de la construcción.

Utiliza el patrón Builder para construir árboles con el patrón Composite u otros objetos complejos.
El patrón Builder te permite construir productos paso a paso. Podrás aplazar la ejecución de ciertos pasos sin descomponer el producto final. Puedes incluso invocar pasos de forma recursiva, lo cual resulta útil cuando necesitas construir un árbol de objetos.
Un constructor no expone el producto incompleto mientras ejecuta los pasos de construcción. Esto evita que el código cliente extraiga un resultado incompleto.

Patrones de creación - **Builder**

Pros

Puedes construir objetos paso a paso, aplazar pasos de la construcción o ejecutar pasos de forma recursiva.

- Puedes reutilizar el mismo código de construcción al construir varias representaciones de productos.
- *Principio de responsabilidad única*. Puedes aislar un código de construcción complejo de la lógica de negocio del producto.

Contras

- La complejidad general del código aumenta, ya que el patrón exige la creación de varias clases nuevas.

- Builder
  - clase abstracta para crear productos.
- Concrete Builder
  - implementación del Builder
  - construye y reúne las partes necesarias para construir los productos
- Director
  - construye un objeto, un producto concreto, usando el patrón Builder configurando el Concrete Builder a usar.
- Producto
  - El objeto complejo bajo construcción